Continuous-Time Reward Machines

By Amin Falah, Shibashis Guha, and Ashutosh Trivedi

Falah, A., Guha, S., & Trivedi, A. (2025). Continuous-time reward machines. Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence. doi:10.24963/ijcai.2025/563

Introduction

Reinforcement Learning (RL)

Motivation

A major challenge in RL is designing reward functions that capture complex task objectives.

Reward Machines (RMs): Non-Markovian rewards using finite-state machines.

Motivation

Classic RL assumes discrete time steps, but real-world systems feature actions with variable delays.

Modeled as Continuous-Time Markov Decision Processes (CTMDPs).

Problem Context

Model-free RL for CTMDPs exists...

Why not use high-quality rate information?
(Ex. historical traffic data)

Related Work

Structured Rewards in Discrete Time

Trying to specify rewards in Discrete-Time MDPs (DTMDPs).

RMs exploit reward function structure to reduce sample complexity.

Related Work

Continuous-Time RL

RL for CTMDPs first introduced by Bradtke and Duff in 1994.

Related Work

No prior work has adapted reward machines specifically for continuous-time settings.

Contributions

1. Continuous-Time Reward Machines (CTRMs)
2. CTMDP System Modeled as DTMDP and Objectives Specified via CTRMs
3. Counterfactual Experiences
4. Novel Reward Shaping Scheme for CTRMs
5. Continuous-Time Benchmarks

Problem Statement

A CTMDP is formally defined as $\mathcal{M} = (S, s_0, Act, P, \lambda, AP, L)$ The key difference from discrete MDPs is the exit rate function $\lambda(s, a)$ meaning the time spent in a state is governed by an exponential distribution.

Problem Statement

The agent seeks to maximize the expected discounted reward, which combines discrete rewards for transitions and continuous rewards accrued over time, discounted by a factor of $e^{-\alpha t}$

Problem Statement

Estimating transition probabilities is difficult, whereas timing rates are often accessible from historical data.

Decompose a CTMDP into a DTMDP paired with a finite-state automaton encoding the rate information.

Problem Statement

A Continuous-Time Reward Machine (CTRM) is defined as

$$\mathcal{R}_C = (\mathcal{U}, u_0, \mathcal{F}, \delta_u, \delta_r, \delta_c, \beta_r)$$

It extends standard RMs by adding a continuous reward function (δ_c) and a rate function (β_r) that specifies timing behavior conditioned on the CTRM state.

Approach

The authors test settings where

1. The CTRM and environment are entirely unknown
2. The CTRM structure is known but rates are unknown
3. The complete CTRM is fully known.

Methods

1. Baseline Q-Learning
2. Counterfactual Experience RL
3. CTRM Reward Shaping

Methods

Baseline Q-Learning

Applying standard CTMDP Q-learning

Treats system (CTRM) as black box

Methods

Counterfactual Experience RL

Leverage known structure of the CTRM

Generates counterfactual updates for all other states in the CTRM

If rates are unknown, approximate via sampling

Methods

CTRM Reward Shaping

Shaping functions for both scalar and continuous rewards (Breadcrumbs)

Formally proves that these additional rewards preserve optimal policies

Results
(Original)

The methods were evaluated on time-critical gridworlds, including an autonomous vehicle in traffic, firefighter and traffic coordination, and a treasure hunt.

Results (Original)

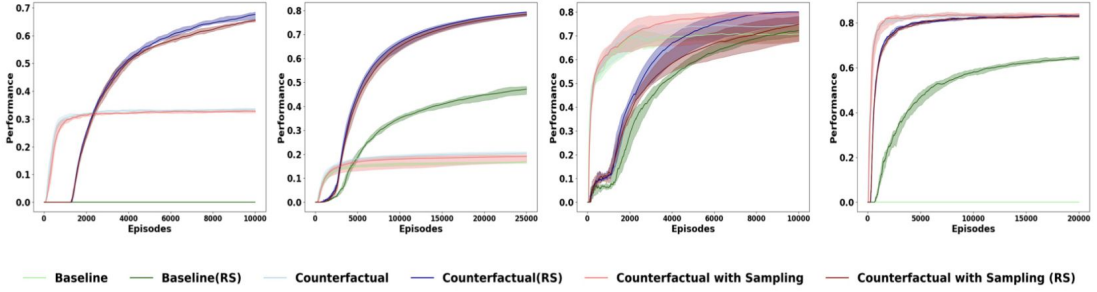
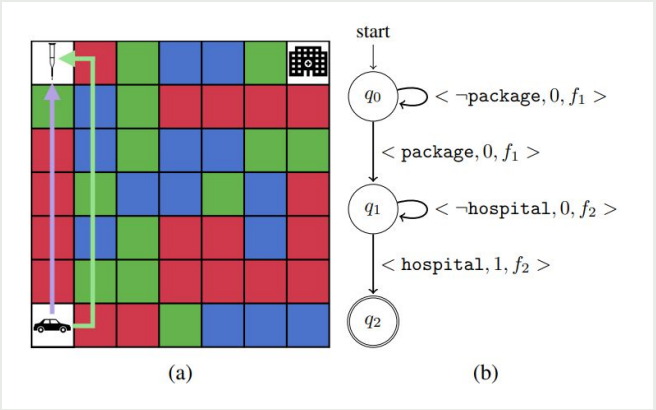


Figure 2: Comparison of the three reinforcement learning approaches—Classic, Counterfactual, and Counterfactual with Sampling—across the following benchmarks: (a) autonomous vehicle in an urban environment, (b) firefighter and traffic coordination, (c) synchronized firefighter and traffic system, and (d) treasure hunt environment.

- Counterfactual approaches typically outperformed the baseline RL method
- Counterfactual RL method using sampled rates performed comparably to the version with full prior rate information
- Reward shaping played a pivotal role
- Depends on the benchmark environments

Results
(Re-implementation)



Benchmark: Autonomous vehicle in an urban environment

Results
(Re-implementation)

1) Autonomous Vehicle in Urban Environment. This benchmark, introduced in Section 1, models an autonomous vehicle navigating a stochastic gridworld (121 states) with varying traffic rates. The CTRM (3 states) encodes the objective of collecting a package and delivering it to a hospital. As shown in Figure 2(a), the baseline RL approach struggles, even with reward shaping. Counterfactual RL methods perform significantly better, with reward shaping further accelerating convergence. Notably, counterfactual RL with sampling performs comparably to the full-information variant, despite lacking rate information.

Results
(Re-implementation)

```python
class AutonomousVehicleEnv:  13 usages
    def __init__(self, rows=11, cols=11, probability=0.9):
        self.rows = rows
        self.cols = cols
        self.probability = probability

        # Special locations
        self.start_pos = (0, 0)  # Distribution center (bottom-left)
        self.package_pos = (10, 0)  # Medical supply depot (bottom-right corner)
        self.hospital_pos = (10, 10)  # Hospital (top-right corner)

        assert 0 <= self.package_pos[0] < cols and 0 <= self.package_pos[1] < rows, \
            f"Package position {self.package_pos} out of bounds!"
        assert 0 <= self.hospital_pos[0] < cols and 0 <= self.hospital_pos[1] < rows, \
            f"Hospital position {self.hospital_pos} out of bounds!"

        self.initstate = (0, 0, 0, 0)
        self.state = self.initstate

        # Actions: 0:north, 1:east, 2:west, 3:south
        self.actions = [0, 1, 2, 3]

        self.env = GridWorld(rows, cols, probability)

        self.statesize = 4  # (x, y, has_package, at_hospital)
        self.actionsize = 4
        self.states = rows * cols  # Total grid positions

        self.time = 0
```

Results
(Re-implementation)

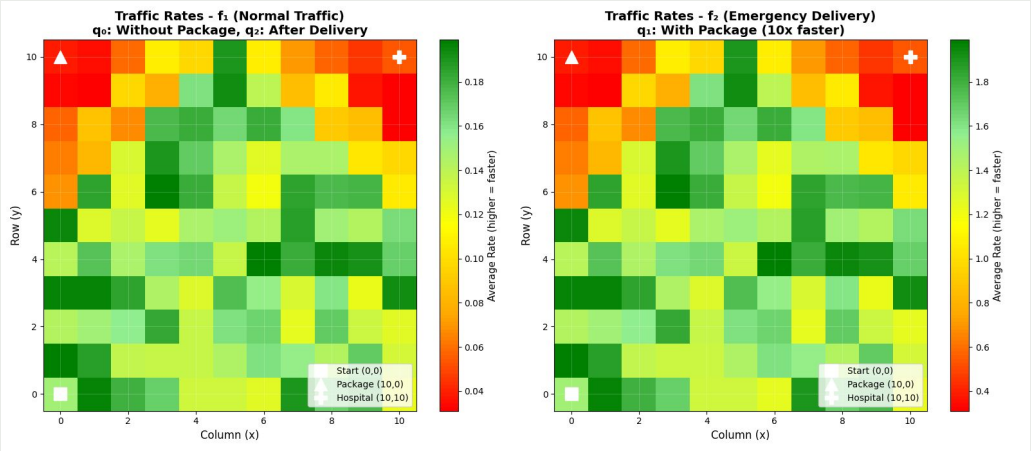
```
class AutonomousVehicleCTRM:  15 usages
    def __init__(self, grid_size=11):
        self.states = [0, 1, 2]  # q0, q1, q2
        self.actions = [0, 1, 2, 3]  # 0:north, 1:east, 2:west, 3:south
        self.totalstates = 3
        self.initstate = 0  # Start in q0 (no package)
        self.state = self.initstate
        self.grid_size = grid_size

        # Generate base rates for the gridworld
        self.base_rates = self.generate_base_rates()

        # f1: Normal traffic rates (for q0 and q2)
        self.function1 = self.create_rate_function(self.base_rates, multiplier=1.0)

        # f2: Emergency delivery rates (for q1 - with package, faster traversal)
        # Emergency vehicles with sirens can traverse faster (10x multiplier)
        self.function2 = self.create_rate_function(self.base_rates, multiplier=10.0)
```

Results
(Re-implementation)



Results
(Re-implementation)

```
class DynamicQLearning:  10 usages
    def __init__(self, alpha=0.01, gamma=0.001, epsilon=0.1, UPDATE_FREQUENCY=50, environment=None, ctrm=None,
                 decay_rate=0.05, reward_shaping=False):
        self.q_table = {}
        self.alpha = alpha
        self.gamma = gamma
        self.max_epsilon = epsilon
        self.min_epsilon = 0.1
        self.decay_rate = decay_rate
        self.epsilon = self.max_epsilon
        self.UPDATE_FREQUENCY = UPDATE_FREQUENCY
        self.evaluation_results = [0]
        self.epsilon_decay = decay_rate
        self.env = environment
        self.ctrm = ctrm
        # Information for checking the value of the strategy
        self.V = {}
        self.ctrmV = {}
        self.states = self.fill_states()
        self.ctrm_states = tuple(self.ctrm.states)
        self.reward_shaping = reward_shaping
        self.fill_vtable()
        self.fillctrmV()
```

Results
(Re-implementation)

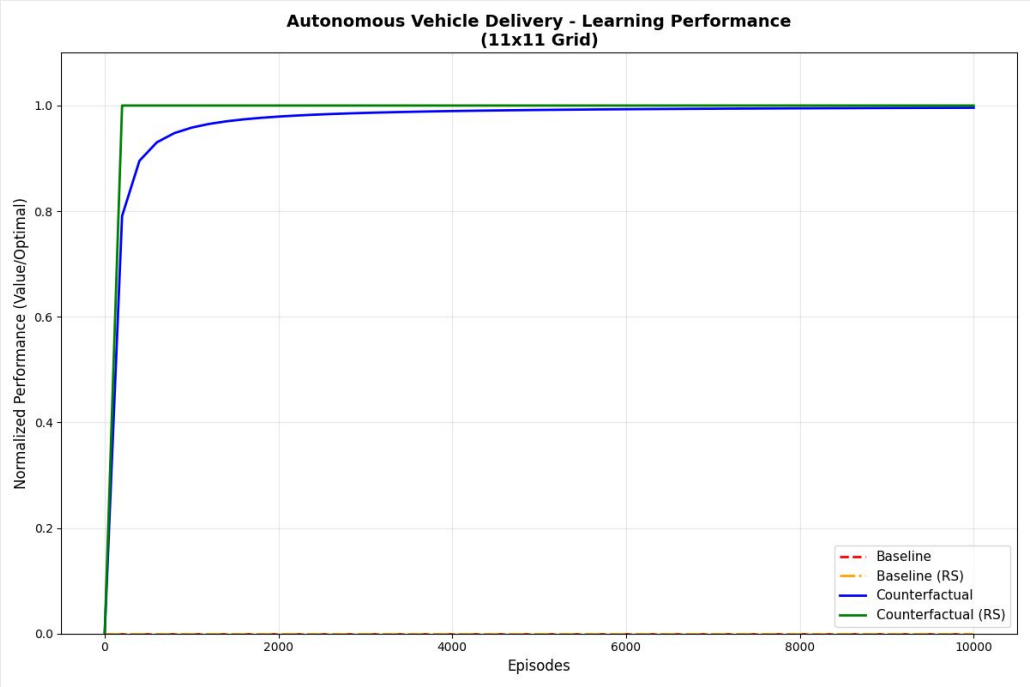
```
class DynamicQLearningCounterFactual:  7 usages
    def __init__(self, alpha=0.1, gamma=0.001, epsilon=0.1, UPDATE_FREQUENCY=50, environment=None, ctrm=None,
                 decay_rate=0.05, reward_shaping=False):
        self.q_table = {}
        self.alpha = alpha
        self.gamma = gamma
        self.max_epsilon = epsilon
        self.min_epsilon = 0.1
        self.decay_rate = decay_rate
        self.epsilon = self.max_epsilon
        self.UPDATE_FREQUENCY = UPDATE_FREQUENCY
        self.evaluation_results = [0]
        self.epsilon_decay = decay_rate
        self.env = environment
        self.ctrm = ctrm
        self.reward_shaping = reward_shaping
        # Information for checking the value of the strategy
        self.V = {}
        self.ctrmV = {}
        self.states = self.fill_states()
        self.ctrm_states = tuple(self.ctrm.states)
        self.fill_vtable()
        self.fillctrmV()
```

Results
(Re-implementation)

```
class DynamicQLearningCounterFactualSampling:  3 usages
    def __init__(self, alpha=0.1, gamma=0.001, epsilon=0.1, UPDATE_FREQUENCY = 50,
                 environment = None, ctrm = None, decay_rate = 0.05, sampling = 10, reward_shaping = False):
        self.q_table = {}
        self.alpha = alpha
        self.gamma = gamma
        self.max_epsilon = epsilon
        self.min_epsilon = 0.1
        self.decay_rate = decay_rate
        self.epsilon = self.max_epsilon
        self.UPDATE_FREQUENCY = UPDATE_FREQUENCY
        self.evaluation_results = [0]
        self.epsilon_decay = decay_rate
        self.env = environment
        self.ctrm = ctrm
        self.sampling = sampling
        self.reward_shaping = reward_shaping
         # Information for checking the value of the strategy
        self.V = {}
        self.ctrmV = {}
        self.states = self.fill_states()
        self.ctrm_states = tuple(self.ctrm.states)
        self.fill_vtable()
        self.fillctrmV()

    def fillctrmV(self):   1 usage
        if self.reward_shaping:
            for ctrm_state in self.ctrm_states:
                self.ctrmV[ctrm_state] = 0 #for CTRM value iteration
```

Results
(Re-implementation)



Conclusion

CTRM
Counterfactual RL
Continuous-time Reward Shaping

Thank You!

$$\mathcal{Q}^{(k+1)}(s,a) := (1-\theta_k)\mathcal{Q}^{(k)}(s,a) + \theta_k\Big(\mathrm{rew}(s,a,s') + e^{-\alpha\tau}\max_{a'}\mathcal{Q}^{(k)}(s',a')\Big), \quad (1)$$

$$\mathcal{Q}^{(k+1)}(s,a) := (1-\theta_k)\mathcal{Q}^{(k)}(s,a) + \theta_k\Big(r(s,a) + \frac{\lambda(s,a)}{\lambda(s,a)+\alpha}\max_{a'}\mathcal{Q}^{(k)}(s',a')\Big). \quad (2)$$

$$\mathcal{M} = (\mathsf{S}, s_0, \mathsf{Act}, P, \mathbf{AP}, L)$$

$$\mathcal{M} = (\mathsf{S}, s_0, \mathsf{Act}, P, \lambda, \mathbf{AP}, L),$$

$$\mathcal{R}_C = (\mathcal{U}, u_0, \mathcal{F}, \delta_u, \delta_r, \delta_c, \beta_r)$$

$$F_S(s,a,\tau,s') = \gamma(s,a,\tau)\cdot\phi_S(s') - \phi_S(s),$$

$$F_C(s,a,\tau,s') = \frac{1}{\omega(s,a,\tau)}\big(\gamma(s,a,\tau)\cdot\phi_C(s') - \phi_C(s)\big),$$

$$((\mathsf{S}', s_0', \mathsf{Act}', P', \lambda, L', \mathbf{AP}'), rew_D, rew_C)$$

$\mathsf{S}' = \mathsf{S}\times\mathcal{U}$; $s_0' = (s_0, u_0)$; $\mathsf{Act}' = \mathsf{Act}$;

$P'((s,u),a,(s',u'))$ equals $P(s,a,s')$ if $u \in \mathcal{F}$ and $u' = u$, or $u \notin \mathcal{F}$ and $u' = \delta_u(u, L(s))$, and equals 0, otherwise;

$L'(s,u) = L(s)$; $\mathbf{AP}' = \mathbf{AP}$;

exit rate $\lambda((s,u),a)$ equals $\beta_r(u)(s,a)$ if $a \in \mathsf{Act}'(s)$, and is 0 otherwise; and

$$rew_D((s,u),a,(s',u')) = \delta_r(u)(s,a,s'), \text{ and}$$
$$rew_C((s,u),a) = \delta_c(u)(s,a).$$

$$(s,u,a,\tau,r,s',u'),$$

s' is the next state observed when taking action a from state s in $\mathcal{M}$,

u' is the next state in the CTRM, determined by the transition $\delta_u(u, L(s))$,

τ is the sampled time, based on the rate $\beta_r(u)(s,a)$, and

r is the reward, computed as

$$r = \delta_r(u)(s,a,s') + \tau\cdot\delta_c(u)(s,a).$$

$$CE(s,a,s') = \bigcup_{u\in\mathcal{U}}\{(s,u,a,\tau,rew(s,a,s'),s',\delta_u(u,L(s)))\},$$

where $rew(s,u,a,s') = \delta_r(u)(s,a,s') + \frac{\delta_c(u)(s,a)}{\beta_r(u)(s,a)+\alpha}$.